ML Project

Numerical Dataset:

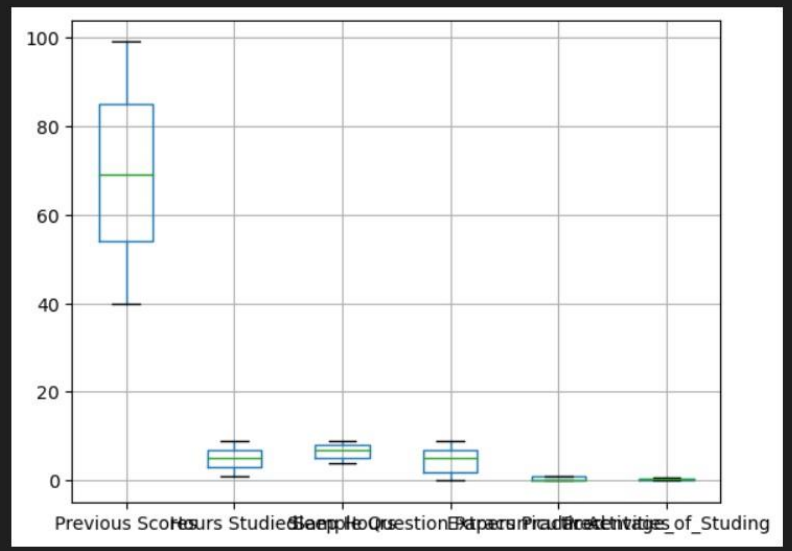
- Dataset in regression model to predict the "Performance Index" based on various features in the dataset.
- Dataset name : Student_performance.
- Number of features: 6 features.
- One target is performance.

Preprocessing:

- Checked for missing values (null values) in the dataset.
- Encoded the "Extracurricular Activities" column by replacing 'No' with 0 and 'Yes' with 1.
- Check outliers.

```
df[['Previous Scores','Hours Studied','Sleep Hours','Sample Question Papers Practiced','Extracurricular Activities','Precentage_of_Studing'
plt.show() #no outliers
```

Python



- **Loading and Exploring Data:**
- You loaded the dataset from a CSV file using Pandas.
- Checked the columns and displayed the first few rows of the dataset.
- Created a new feature "Precentage_of_Studing" based on "Hours Studied" and "Sleep Hours".
- **Linear Regression :**
- Trained a Linear Regression model and evaluated its performance.

```
#split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.32, random_state=29, shuffle =True)
```

```
#
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
#train the LinearRegression model
linearregression = LinearRegression()
linearregression.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
y_pred=linearregression.predict(X_test)
y_pred[:5]
```

- **K-Nearest Neighbors (KNN):**
- Used GridSearchCV to find the best hyperparameters for the KNN model.

```python
#Use grid search and choose best Hyperparameters
param_grid = {'n_neighbors': list(range(1, 21)) ,'weights':('uniform','distance')}  #Define the parameter grid
# Create a KNN regressor
knn_regressor = KNeighborsRegressor()
# Use grid search with 10 cross-validation
grid_search = GridSearchCV(knn_regressor, param_grid, cv=10)
grid_search.fit(X_train, y_train)
```

Python

- Evaluated the model performance using metrics and regression plot.

```
train_score = grid_search.score(X_train, y_train)
test_score =grid_search.score(X_test, y_test)
print(f"Train_score is {train_score} \nTest_score is {test_score} ")
```

```
Train_score is 0.9993605434113016
Test_score is 0.9791379907380208
```

```
y_pred = grid_search.predict(X_test)
print(f"First 5 values in y pred is {y_pred[:5]} \nFirst 5 values in y test is {y_test[:5]} ")
```

```
First 5 values in y pred is [68.28281198 75.65916077 21.74766773 73.4165835  65.00316672]
First 5 values in y test is [67. 76. 21. 72. 65.]
```

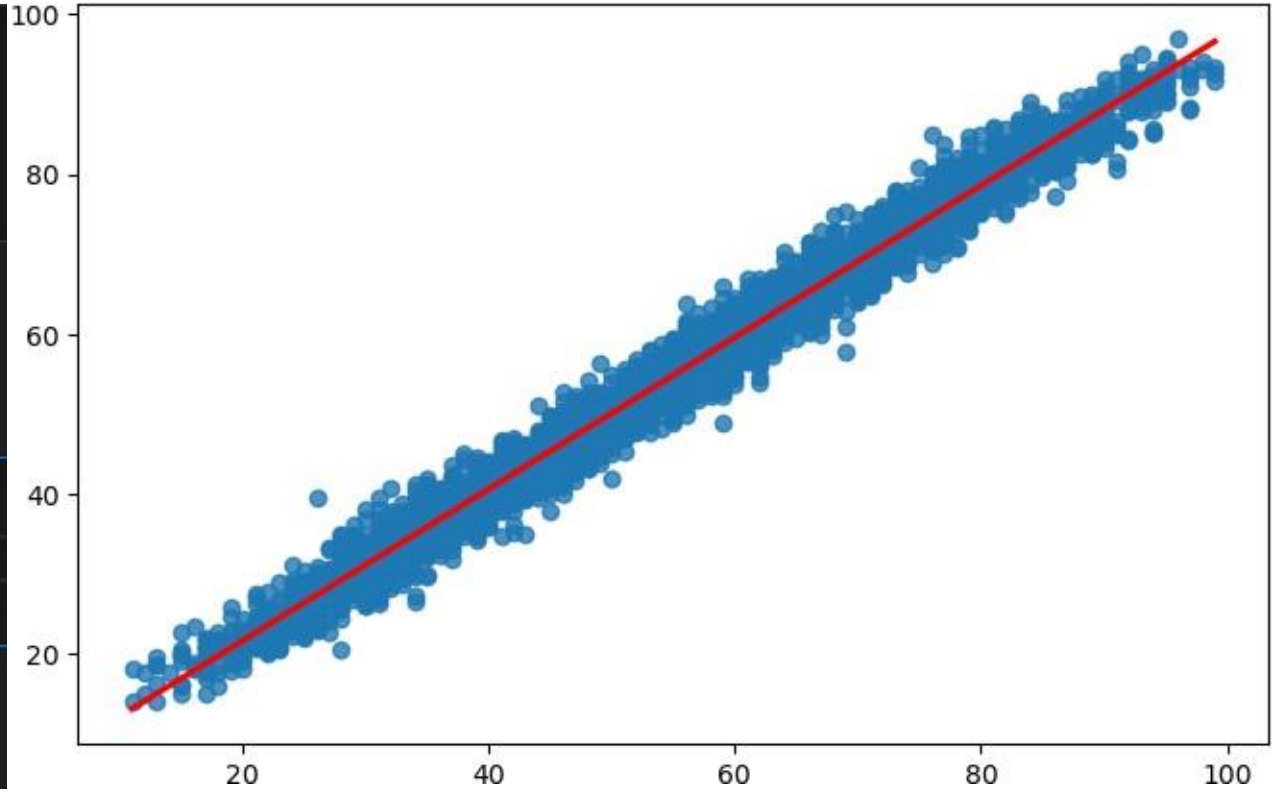


Image Dataset :

Dataset name: **Oxford-IIIT Pet Dataset.**

Dataset link: [Visual Geometry Group - University of Oxford]

- Classes : cats and dogs.
- Number of samples: 3686 images.
- Number of samples in testing: 738 images.
- Number of samples in training: 2948 images

Logistic Regression Model:

- Feature Extraction Phase :
- Number of Features : Length of the HOG descriptor.
- Names of Features: HOG features.
- Dimension of Resulted Features: Determined by the length of the HOG descriptor.
- **Data Splitting :**
- **Training Set:** 80% of the data
- **Testing Set:** 20% of the data

```python
#labels : cat - dog
from skimage import color , exposure ,io , transform
import pathlib
import xml.etree.ElementTree as ET
from skimage.filters import gaussian
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import silhouette_score
def read_voc_xml(xmlfile: str) -> dict:
    """read the Pascal VOC XML and return (filename, object name, bounding box)
    where bounding box is a vector of (xmin, ymin, xmax, ymax). The pixel
    coordinates are 1-based.
    """
    root = ET.parse(xmlfile).getroot()
    boxes = {"filename": root.find("filename").text,
             "objects": []
             }
    for box in root.iter('object'):
        bb = box.find('bndbox')
        obj = {
            "name": box.find('name').text,
            "xmin": int(bb.find("xmin").text),
            "ymin": int(bb.find("ymin").text),
            "xmax": int(bb.find("xmax").text),
            "ymax": int(bb.find("ymax").text),
        }
        boxes["objects"].append(obj)

    return boxes
```

```python
base_path = pathlib.Path("E:/FCAI/ML/oxford-iiit-pet")
images_src = base_path / "images"
annotation_src = base_path / "annotations" / "xmls"
features=[]
labels=[]
for xmlfile in annotation_src.glob("*.xml"):
    #read image from xml file
    ann = read_voc_xml(str(xmlfile))
    img = io.imread(str(images_src / ann["filename"]))
    bbox = ann["objects"][0]
    # Append the label to the labels list
    labels.append(bbox["name"])
    #pre processing
    grayscale_image = color.rgb2gray(img) if len(img.shape) == 3 else img
    face_image = grayscale_image[bbox["ymin"] : bbox["ymax"], bbox["xmin"] : bbox["xmax"]]
    resized_image = transform.resize(face_image,(294 , 288))
    #enhance contrast for better quality and for lighting regions
    enhanced_image = exposure.equalize_adapthist(resized_image)
    #guassian blur to filter noise
    gaussian_image = gaussian(enhanced_image, sigma=1)
    #feature extraction
    hog_descriptor, hog_image = hog(image= gaussian_image, orientations= 8, pixels_per_cell=(8,8),
        cells_per_block=(1,1), visualize=True, block_norm='L2', feature_vector=True ,channel_axis=None)
    ##for better hog image display
    hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 5))
    fig, axes = plt.subplots(1, 3, figsize=(20, 10))
    axes[0].imshow(img, cmap='gray')
    axes[0].set_title('original image')
    axes[1].imshow(gaussian_image, cmap='gray')
    axes[1].set_title('pre processing image')
    axes[2].imshow(hog_image, cmap='gray')
    axes[2].set_title('HOG image')
    plt.show()
    # Append the descriptor to the features list
    features.append(hog_descriptor) #hog_descriptor is numpy array
```

[1] ✓ 40m 59.8s

```
features_array = np.array(features)
#standardlization
features_array_standard  = StandardScaler().fit_transform(features_array)
#converting label names into numeric labels
numeric_labels = LabelEncoder().fit_transform(np.array(labels))
#split data
X_train, X_test, y_train, y_test = train_test_split(features_array_standard, numeric_labels, test_size=0.2, random_state=42, shuffle=True)
print("total numer of images: ", len(numeric_labels))
print(np.unique(numeric_labels))
```

[32] ✓ 5.0s

```
total numer of images:  3686
[0 1]
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import seaborn as sns
from sklearn.metrics import confusion_matrix,accuracy_score
```

[34] ✓ 0.0s

```
X = features_array
y=numeric_labels
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.22, random_state=42, shuffle=True)
```

[35] ✓ 0.2s

- **Hyperparameters :**

- **Penalty:** L2
- **Solver:** Saga
- **Regularization Parameter (C):** 0.001
- **Random State:** 33
- **Number of Jobs:** Not specified
- **Multi-Class Strategy:** Auto
- **Maximum Iterations:** 100

```python
# Create a logistic regression model
model = LogisticRegression()
# Train the model
model.fit(X_train, y_train)
```

[36] ✓ 7.7s

```python
train_score=model.score(X_train,y_train)
test_score=model.score(X_test,y_test)
print(f"Train_score is {train_score} \nTest_score is {test_score} ")
```

[37] ✓ 0.1s

```
Train_score is 1.0
Test_score is 0.9087546239210851
```

```python
# Make predictions on the test set
y_pred = model.predict(X_test)
y_pred[:10]
```

[39] ✓ 0.0s

```
array([1, 0, 1, 1, 0, 0, 1, 1, 1, 0], dtype=int64)
```

```python
print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
```

[40] ✓ 0.0s

```
[[1 1]
 [0 1]
 [1 1]
 ...
 [1 1]
 [1 1]
 [1 1]]
```

```python
ac=accuracy_score(y_test, y_pred)
print(ac)
```

[41] ✓ 0.0s

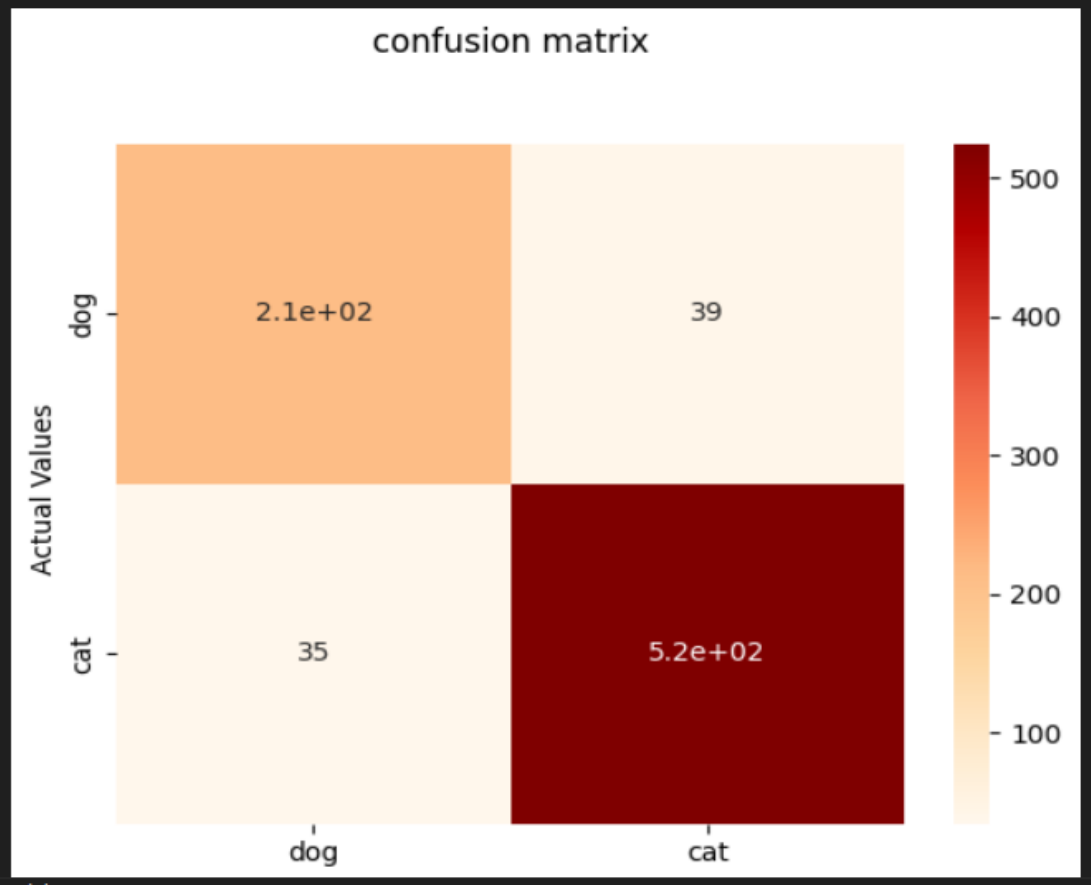
```
0.9087546239210851
```

- **Results on Testing Data:**

```python
print(confusion_matrix(y_test, y_pred))
ax=sns.heatmap(cm, annot=True , cmap='OrRd')
ax.set_title('confusion matrix' +"\n\n")
ax.set_xlabel('\nPredicted Values')
ax.set_ylabel('Actual Values ')
ax.xaxis.set_ticklabels (['dog', 'cat'])
ax.yaxis.set_ticklabels (['dog', 'cat'])
plt.show()
```

[42] ✓ 0.3s

```
[[213  39]
 [ 35 524]]
```



- **Conclusion:**

- The logistic regression model achieved an accuracy of 90.8% on the testing set, as evidenced by the confusion matrix. This indicates a reasonably good performance in classifying images of the pets : cat and dog based on the extracted HOG features. Further analysis and potentially fine-tuning of hyperparameters could be performed to improve the model's performance.

Kmeans Model:

- **Data Splitting :**
- **Training Set:** 80% of the data
- **Testing Set:** 20% of the data

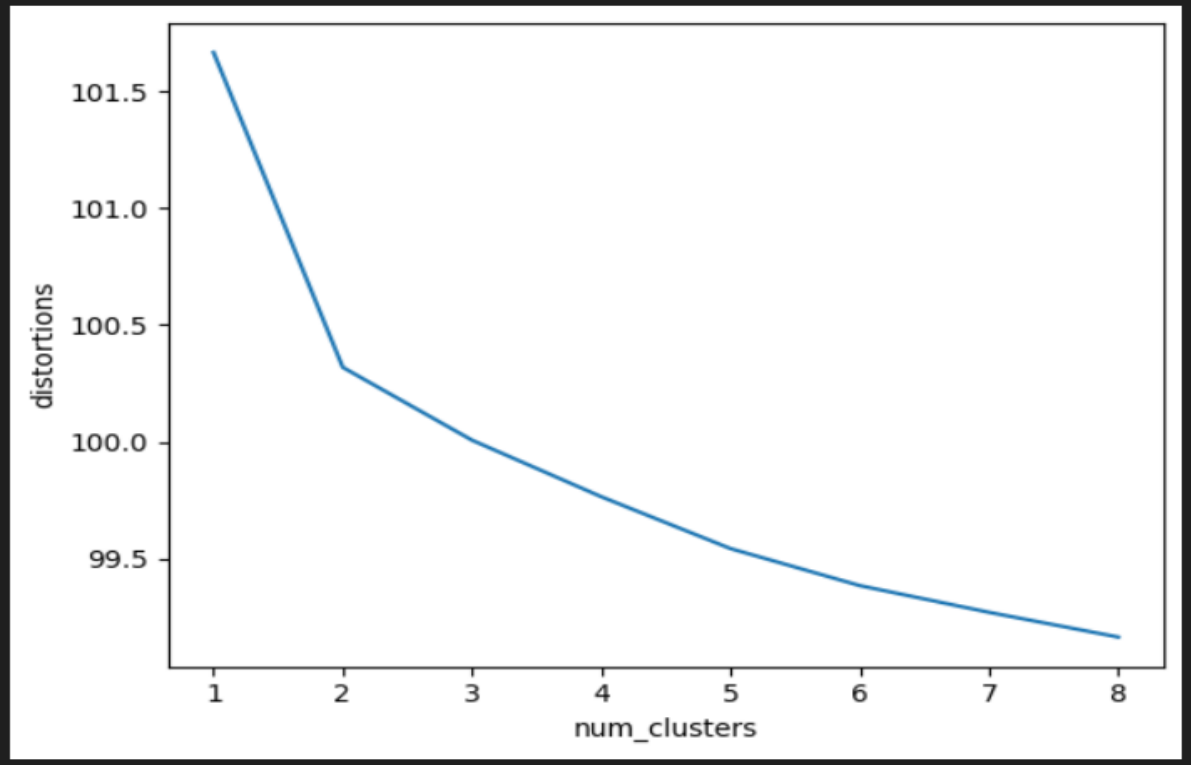
- **Implementation Details:**
- **Clustering Phase:**
  - **Number of Clusters (K):** Determined using the Elbow Method
  - **Initialization Method:** k-means++
  - **Maximum Number of Iterations:** 1000
  - **Tolerance:** 1e-04
  - **Random State:** 42

- **Elbow Method :**

Optimal Number of Clusters (K) = 2

```
from scipy.cluster.vq import kmeans
import pandas as pd
import seaborn as sns
distortions = []
num_clusters = range(1, 9)
for i in num_clusters:
    cluster_centers, distortion = kmeans(X_train, i)
    distortions.append(distortion)
elbow_plot = pd.DataFrame({'num_clusters': num_clusters, 'distortions': distortions})
sns.lineplot(x='num_clusters', y='distortions', data=elbow_plot)
plt.xticks(num_clusters)
plt.show()
```

[104]

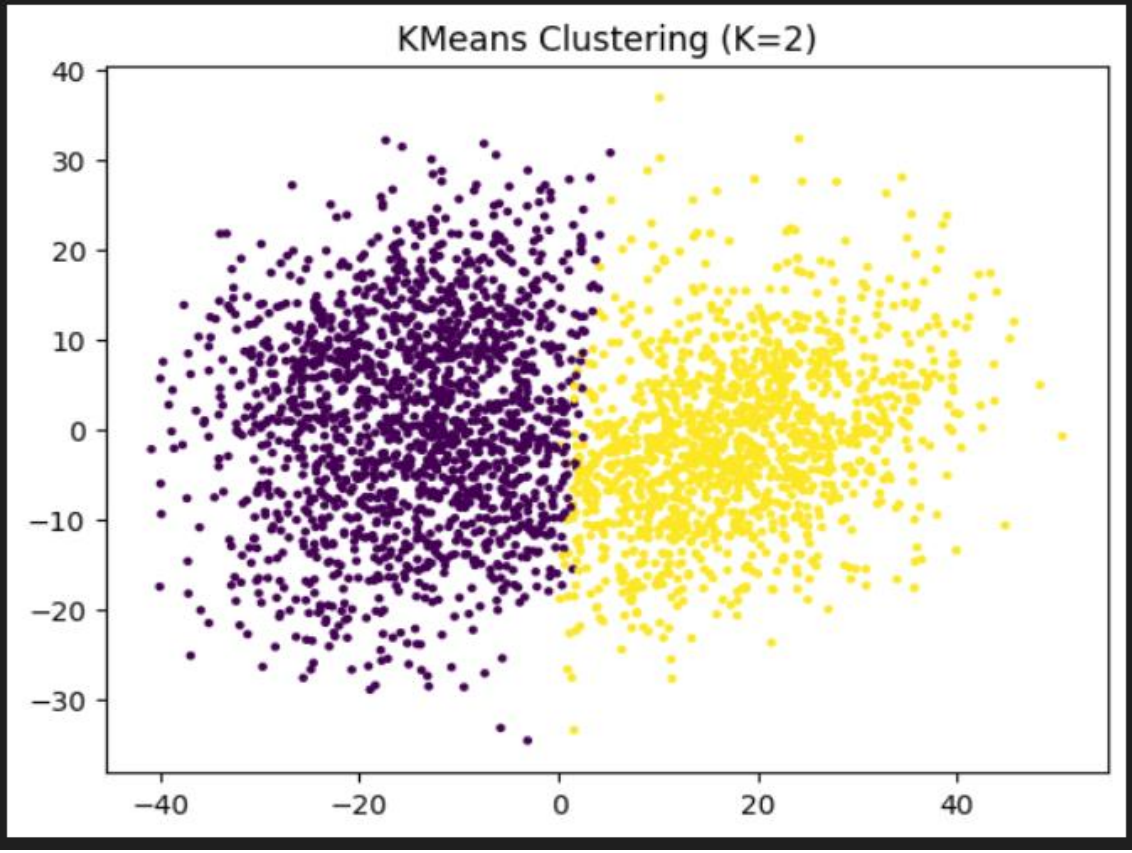


- **Clustering Results for X_train :**

```python
from sklearn.metrics import accuracy_score
kmeans = KMeans(n_clusters=2, init='k-means++', n_init=10, max_iter=1000,  tol=1e-04, random_state=48)
cluster_labels = kmeans.fit_predict(X_train)
print('Train : \nAccuracy : {}'.format(accuracy_score(cluster_labels,y_train)))
#visualization
X_pca = PCA(n_components=2).fit_transform(X_train)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cluster_labels, cmap='viridis', marker='o', s=5)
plt.title(f'KMeans Clustering (K={2})')
plt.show()
```

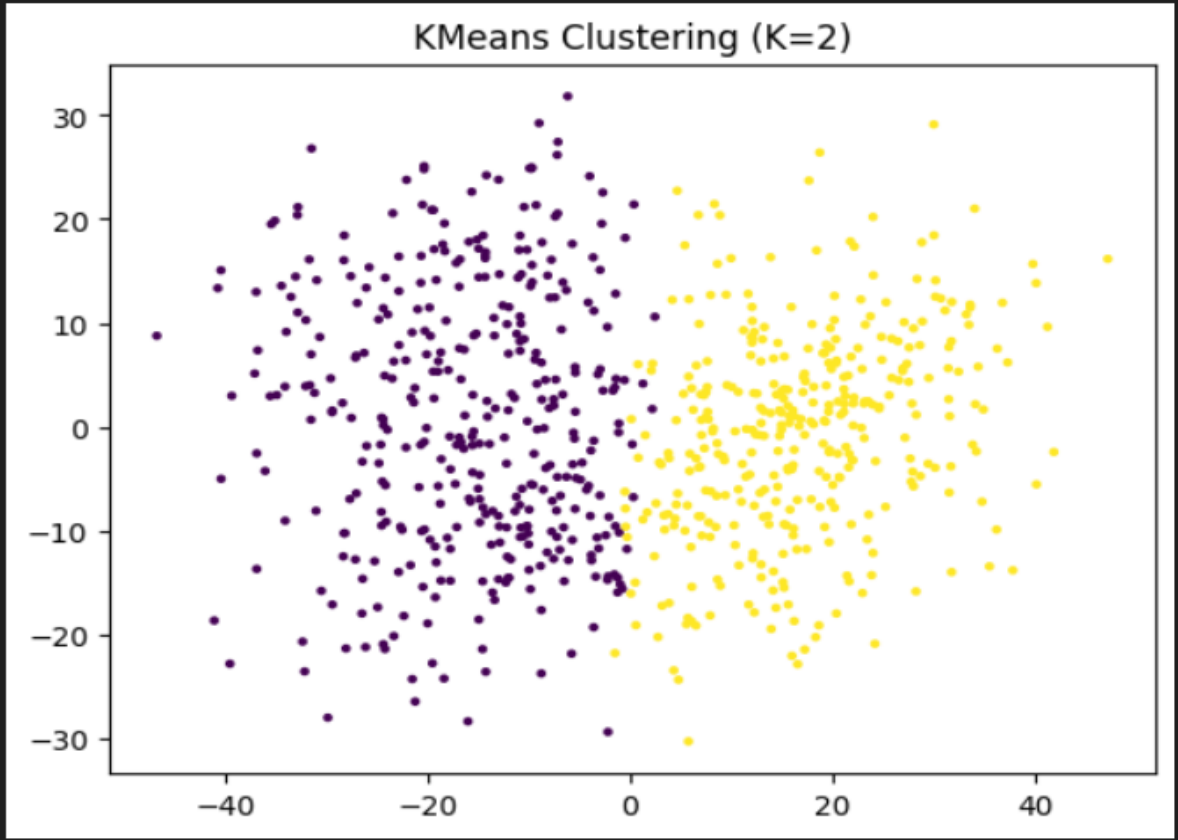
[5] ✓ 9.9s

```
Train :
Accuracy : 0.764246947082768
```



- **Clustering Results for X_test :**

```
cluster_labels = kmeans.predict(X_test)
print('Test : \nAccuracy : {}'.format(accuracy_score(cluster_labels,y_test)))
X_pca = PCA(n_components=2).fit_transform(X_test)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cluster_labels, cmap='viridis', marker='o', s=5)
plt.title(f'KMeans Clustering (K={2})')
plt.show()
```

[11] ✓ 0.8s

```
Test :
Accuracy : 0.7764227642276422
```